

R-BurstHADS: Complete Technical Description

How the scheduler works from first principles, covering the machine pool, the workload, the deadline, the simulation engine, both scheduling phases, every formula, and the billing model.

1. What the system is trying to do

The problem is this. We are given a set of independent tasks and a single deadline by which all of them must finish. We can rent three kinds of machine from a cloud provider. Spot machines are cheap but the provider can suspend them at any moment. Burstable machines are cheap and never suspended, but they run slowly unless they spend a limited supply of accumulated credits. On demand machines are expensive but completely reliable and available in unlimited quantity. The goal is to finish all tasks before the deadline while spending as little money as possible.

R-BurstHADS solves this in two phases. The first phase happens once, before anything runs, and produces a plan mapping every task to a machine. The second phase runs continuously during execution and repairs the plan whenever a spot machine is suspended. Almost all of the interesting behaviour is in the second phase.

2. The three entities

2.1 A task

A task carries an execution time measured in seconds on a reference core, a memory requirement in megabytes, and a deadline inherited from the job stage it belongs to. During execution it also carries bookkeeping: which machine it is assigned to, when it started, whether it has completed, how much work remains, and whether it is currently running in the slow economy mode of a burstable machine.

The single most important field is remaining time. It starts equal to the execution time and is only ever reduced by the checkpoint mechanism described in section 9.3. It is never decremented gradually as the task runs, because in a discrete event simulation nothing happens between events. The passage of work is represented purely by the arithmetic that computes when a task will finish.

2.2 A machine

A machine carries a market type, a speed, a core count, a memory capacity, a price per second, and for spot machines an interruption rate. It also holds two lists: every task assigned to it, and the subset of those actually executing right now. The second list can never be longer than the core count.

Speed and core count are separate quantities and both matter. Speed is how much faster a single task runs on this machine than on a reference core. Core count is how many tasks can run at the same time. The total work a machine retires per second is therefore the product of the two, and this product is the quantity we call throughput.

$$\text{throughput of a machine} = \text{speed} \times \text{cores}$$

Getting this wrong was one of the significant bugs we found. A formula that uses speed alone treats a four core machine as no more capable than a two core machine of the same clock, which is wrong by a factor of two.

2.3 The machine pool

The pool is the catalogue of machines available to every scheduler. All three schedulers draw from exactly the same pool, which is what makes the comparison fair. It contains three copies of each spot type, one burstable machine, and one on demand machine which acts as a template for an unlimited supply.

Type	Market	Speed	Cores	Memory	Price per hour	Interruption rate
c5.large	spot	2	2	4 GB	0.0306	0.03 per hour
c5.xlarge	spot	4	4	8 GB	0.0612	0.04 per hour
m5.xlarge	spot	4	4	16 GB	0.0700	5 per 2700 seconds
t3.large	burstable	2	1	8 GB	0.0832	never interrupted
c5.large	on demand	2	2	4 GB	0.0850	never interrupted

With three copies of each spot type the total spot throughput is three times the sum of two times two, four times four, and four times four, which is one hundred and eight units of work per second. This number drives the deadline calculation.

Note that the burstable machine is forced to a single core regardless of its real specification. This is not a hardware limit but a deliberate rule from the source paper: a burstable machine never holds more than one task at a time, so that it spends as little of its credit reserve as possible and remains available as rescue capacity.

3. Generating the workload

Each task is generated independently. Its execution time is drawn uniformly between one hundred and three hundred and fifty seconds, and its memory requirement uniformly between 2.85 and 13.19 megabytes. These ranges are taken from the reference jobs published in the source paper. The number of tasks is a parameter of the experiment and takes the values ten, twenty, fifty, one hundred, two hundred and three hundred.

Because tasks are generated from a seeded random number generator, and because the generator is reset immediately before generation, all three schedulers in a given run receive an identical set of tasks. This is what makes the comparison paired: any difference in the results is attributable to the scheduler and not to the workload.

4. Deriving the deadline

The deadline cannot be an arbitrary number. If it is too loose no scheduler is ever under pressure and the experiment cannot distinguish them. If it is too tight the problem is simply infeasible. We therefore derive it from what the pool is physically capable of.

First we compute the fastest possible finishing time, assuming perfect packing and no overhead. This has two terms, because a job cannot finish faster than its total work divided by total throughput, and it also cannot finish faster than its single longest task running alone on the fastest machine.

$$M^* = \max(n \times \text{average execution time} / \text{spot throughput} , \text{longest task} / \text{fastest speed})$$

The deadline is then a fixed multiple of that bound.

$$D = \text{deadline factor} \times 3 \times M^*$$

The multiplier of three was chosen by measurement, not by guesswork. We measured what the schedulers actually achieve and found that an undisturbed well organised run lands at between 1.16 and 2.36 times the bound, while a run disrupted by interruptions lands at between 6.6 and 7.6 times. A multiplier of three therefore sits exactly in the gap: an undisturbed run meets it, and a disrupted run does not unless the scheduler buys extra capacity. That is precisely the pressure the experiment needs to apply in order to tell the schedulers apart.

A floor is applied so the deadline never falls below the reserve described next, because below that point the problem is infeasible rather than merely difficult.

4.1 The spot reserve

Spot machines are not scheduled against the full deadline. They are scheduled against a reduced deadline that holds back enough time to rescue the longest task if its machine is suspended at the worst possible moment.

$$D_{\text{spot}} = D - (\text{longest task} \times (1 + \text{overhead}) / \text{slowest machine speed})$$

The reasoning is that if a machine is suspended, its tasks must be moved somewhere else and finished. In the worst case the task moved is the longest one and the only machine available is the slowest one. Holding back that much time guarantees the rescue can always succeed. On demand machines are scheduled against the full deadline instead, because they are never suspended and so need no reserve.

5. How a machine executes tasks

When a machine has a free core and a waiting task, it starts it. The time the task will take is its remaining work divided by the effective speed of the machine, inflated by a ten percent checkpointing overhead.

$$\text{execution time} = (\text{remaining work} / \text{effective speed}) \times 1.10$$

The overhead represents the cost of periodically saving progress so that work is not lost if the machine is suspended. It is charged whenever a task runs. An earlier version charged it only on a task's first attempt and skipped it on any retry, which made recovering from an interruption cheaper than never being interrupted, and that in turn made the whole simulation reward failure.

Effective speed is simply the machine speed for spot and on demand machines. For a burstable machine it depends on the mode the task is running in.

$$\begin{aligned} \text{effective speed of a burstable machine} &= \text{speed} && \text{in burst mode} \\ &= \text{speed} \times 0.20 && \text{in baseline mode} \end{aligned}$$

The machine then creates a completion event scheduled for the current time plus the execution time, and the simulation moves on.

6. CPU credits on burstable machines

A burstable machine models the AWS T3 credit system. It begins with a full balance of one hundred and forty four credits. While a task runs in baseline mode the machine earns credits; while a task runs in burst mode it spends them.

Quantity	Formula	Value used
Earn rate	fixed	36 credits per 3600 seconds
Spend rate	$(1 / 60) \times \text{speed per second}$	0.033 per second at speed 2
Maximum balance	fixed	144 credits
Credits a task needs	$(\text{execution seconds} / 60) \times \text{speed}$	

Before a task is allowed to migrate onto a burstable machine in burst mode, the scheduler checks that the machine holds at least as many credits as the task will require. This is what stops the rescue tier being used beyond its capacity.

7. Interruption risk

Each spot machine carries an interruption rate. The probability that it is suspended during any given interval follows the standard exponential form.

$$\text{probability of interruption} = 1 - e^{(- \text{rate} \times \text{duration})}$$

This is used in two places. It drives the preemptive provisioning decision described in section 8.5, and it generates the actual interruption events in the experiment. In the experiments the rate is set from the scenario definitions of the source paper, where the rate equals an expected count divided by the deadline.

Scenario	Expected interruptions	Expected recoveries
sc1	1	0
sc2	5	0
sc3	1	5
sc4	5	5
sc5	3	2.5

Each spot machine is subject to its own independent process, so interruptions are individual random events rather than a single simultaneous outage.

8. Phase one: building the initial plan

This phase runs once, at time zero, and produces a complete assignment of tasks to machines. It has five steps.

8.1 Ranking the machines

Candidate machines are ranked by how much computing power each rupee buys. This is the formula the source paper calls Equation 8.

$$\text{weight of a machine} = (\text{speed} \times \text{cores}) / \text{price per second}$$

Selection then uses smooth weighted round robin rather than always taking the best machine. Each candidate keeps a running counter. On every selection every counter increases by that machine's weight, the machine with the highest counter wins, and the winner's counter is reduced by the sum of all weights. Over many selections this visits machines in proportion to their weight instead of piling everything onto whichever one looks best at that instant.

8.2 The greedy initial solution

Tasks are sorted by memory requirement, largest first, and placed one at a time. For each task the scheduler tries four things in order. First, any machine already opened, least loaded first, checked against the spot reserve. Second, a brand new spot machine chosen by the weighted round robin above. Third, an on demand machine already running, cheapest first, checked against the full deadline. Fourth, a brand new on demand machine drawn from the unlimited supply.

If even a brand new empty on demand machine cannot finish the task before the deadline, the scheduler raises an error rather than continuing. This last step was missing in our implementation and is the bug described in the change report, where tasks that fitted nowhere were silently dropped from the plan entirely.

A placement is feasible if the machine has enough free memory and if list scheduling the machine's tasks across its cores finishes before the applicable deadline.

8.3 Local search and the iterated search around it

The initial solution is then improved. A single local search pass picks one destination machine at random and repeatedly tries moving a random ten percent of tasks onto it, keeping whichever attempt scores best. The outer loop runs two hundred iterations, adding one unused spot machine at random each time so the search has somewhere new to spread into.

Solutions are scored by a fitness function combining cost and finishing time, both normalised to lie between zero and one, with equal weight.

$$\begin{aligned} \text{fitness} &= 1.0 \text{ if makespan exceeds the bound} \\ &= 0.5 \times \text{normalised cost} + 0.5 \times \text{normalised makespan} \quad \text{otherwise} \end{aligned}$$

The value of one is an infeasibility penalty. Because both normalised terms lie between zero and one by construction, one is already worse than any feasible solution, so it acts as a sentinel without needing to be infinite.

When the search stops improving it is allowed to relax its bound by twenty five percent so it can escape a local optimum. This relaxation is applied at most once per stalled window and is capped at the real deadline. Without the cap it compounded on every iteration and grew to astronomic values, which removed the constraint entirely.

8.4 Allocating burstable machines

After the search converges, a number of burstable machines are launched in proportion to how many machines the search selected.

$$\text{number of burstable machines} = \text{ceiling}(0.2 \times \text{machines selected})$$

The ceiling matters. With ordinary rounding, a search that selected two machines would launch zero burstable machines and the entire rescue mechanism would silently not exist.

Tasks projected to overrun the spot reserve are then moved onto these machines in baseline mode, but only when the move is genuinely an improvement. Since baseline mode runs at a fifth of normal speed, moving a task that did not need moving makes it five times slower, and we measured exactly that turning a task of three hundred and forty eight seconds into one of nine hundred and fifty seven seconds.

8.5 Preemptive provisioning, the first of our two additions

Finally the scheduler looks at every spot machine carrying real interruption risk and asks whether it is cheaper to hold a replacement ready in advance than to rescue the machine's tasks after the fact. It compares two quantities.

$$\text{reactive cost} = \text{tasks on the machine} \times \text{average task time} \times (\text{on demand rate per unit speed} - \text{spot rate per unit speed})$$
$$\text{proactive cost} = \text{startup time} \times \text{spot rate} + (\text{average task time} / \text{spot speed}) \times \text{spot rate}$$

A replacement is launched only when the risk weighted reactive cost exceeds the proactive cost.

$$\text{launch a replacement if } \text{probability of interruption} \times \text{reactive cost} > \text{proactive cost}$$

The replacement is timed to become ready before the interruption it anticipates, and its type is chosen as the most cost efficient spot type present in the pool, using the same ranking formula as section 8.1. It is important that the type is resolved from the pool rather than hard coded, because a hard coded type is how our scheduler ended up able to obtain a machine the baselines could not.

9. Phase two: reacting to interruptions

The simulation is driven by an event queue. Events are held in a list which is sorted by time before each dispatch, the earliest is removed and executed, and simulation time jumps to that event's timestamp. Events created during execution are picked up on the next pass. The run ends when the queue is empty.

9.1 What happens when a task finishes

The completion event first checks that it has not been superseded, which is how migrations cancel obsolete events. It then settles CPU credits if the machine is burstable, marks the task complete, releases its memory, removes it from both machine lists, and immediately tries to start the next waiting task in the freed core. If no tasks remain the machine becomes idle and starts a termination timer. Finally work stealing is attempted.

9.2 What happens when a machine is suspended

This is the central mechanism of the whole system. The machine is marked suspended and billing stops immediately, which reflects the real provider behaviour that a suspended instance is not charged for compute. Every unfinished task on it, whether running or merely queued, is collected, checkpointed, and passed one at a time to the machine selection cascade described next.

9.3 Checkpointing

Before a task is moved, the work it has already completed is recorded so that it does not restart from the beginning.

$$\text{work completed} = \text{elapsed time on this machine} \times \text{machine speed}$$

$$\text{remaining work} = \max(0, \text{previous remaining work} - \text{work completed})$$

When the task is placed on its new machine, its remaining work is restored from that record. A task that had not yet started keeps its full remaining work, since there is nothing to save.

9.4 The migration cascade

For each displaced task the scheduler tries five things in strict order, stopping at the first that works. The first and fourth are our additions; the others are inherited unchanged from the source paper.

Order	Destination	Why it is tried at this point
1	A replacement machine provisioned in advance	It exists specifically to absorb this rescue, so it is tried before anything else
2	An idle burstable machine with enough credits	Cheap, already running, and needs no startup delay
3	Any active machine, spot before on demand, idle before busy	Uses capacity already paid for
4	Newly provisioned spot capacity, sized by the deadline	Replaces lost spot capacity with spot capacity rather than expensive on demand
5	A brand new on demand machine	Guaranteed to be available, but the most expensive option

Every candidate must pass a feasibility test: it needs enough free memory, and the task must be projected to finish before the deadline there. Spot destinations face an additional test requiring spare time greater than the longest task already queued on them, so a rescue does not land on a machine which itself has no room to recover.

9.5 Deadline aware provisioning, the second of our two additions

This is the step that replaced a fixed constant and is the substantive algorithmic contribution. The previous rule provisioned enough machines that each held at most eight tasks. That number took no account of how much time remained, so it bought the same capacity whether the deadline was minutes or hours away.

The replacement is a conservation of work argument. We ask how much work is waiting, how much usable time remains once a new machine has finished starting up, and how much throughput is already available.

$$\begin{aligned} \text{usable time} \quad T &= \text{deadline} - \text{current time} - \text{startup latency} \\ \text{required rate} &= \text{waiting work} / T \\ \text{shortfall} &= \text{required rate} - \text{throughput already active} \\ \text{machines to buy} &= \text{ceiling}(\text{shortfall} / \text{throughput of one machine}) \end{aligned}$$

If the shortfall is zero or negative, nothing is bought and the scheduler behaves exactly like its parent. This is why our method is inert when the deadline is comfortable, and it is why it costs nothing in situations where it is not needed.

No safety margin is added to this estimate. If it turns out optimistic, the per task placement path adds capacity incrementally under its own deadline test, so the mechanism is self correcting and does not need a fudge factor.

9.6 Work stealing

Whenever a machine has a free core, it may pull not yet started tasks from machines that still have a backlog. Expensive on demand machines are drained first, so costly capacity can be released sooner. Only queued tasks are eligible, never running ones, so no work in progress is ever discarded. If the destination is a burstable machine, exactly one task is moved and it runs in baseline mode, which earns credits rather than spending them.

9.7 Resumption and idle termination

If the scenario models recovery, a suspended machine can come back. It returns to idle, billing restarts, and it attempts to steal work, because its own former tasks were reassigned when it was suspended and are not returned to it.

Separately, a machine that has been idle for nine hundred seconds is terminated to stop it accruing charges. Burstable machines are exempt, because their value is precisely that they sit idle accumulating credits ready for the next rescue.

10. Billing and measurement

Billing follows the provider model rather than counting task time. Each machine records intervals during which it was deployed. An interval opens when the machine first starts a task, or when it is provisioned, or when it resumes. It closes when the machine is suspended or terminated. An interval still open at the end of the run is closed at the makespan.

$$\begin{aligned} \text{seconds billed} &= \text{sum over intervals of (end - start)} \\ \text{total cost} &= \text{sum over all machines of (price per second x seconds billed)} \end{aligned}$$

The consequence worth stating plainly is that idle but running machines are charged for. This is what makes over provisioning visibly expensive, and it is why the deadline aware sizing rule of section 9.5 produces a real saving rather than an accounting artefact. We verified that machines provisioned during a run are included in this total. At three hundred tasks under frequent interruption they account for about eighty five percent of the bill, so the cost of our own mechanism is fully counted against it.

Makespan is the latest finishing time across all tasks. A deadline miss is any task finishing after its deadline. Both measures consider only tasks that actually completed, which is why the silent task dropping bug mattered so much: a dropped task contributed neither cost, nor time, nor a recorded failure. It simply vanished. With that bug fixed, every task completes in every configuration, so the measures are now computed over the full workload.

11. A complete run, end to end

It is worth tracing one run in sequence to see how the pieces connect.

At time zero the pool is built and the tasks are generated. The deadline is computed from the pool throughput. The scheduler computes the spot reserve, builds a greedy initial plan, improves it with two hundred iterations of local search, launches burstable machines in proportion to the plan, moves any projected late tasks onto them if that helps, and finally decides whether to hold any replacement spot machines in reserve. Every machine then starts as many tasks as it has cores, each creating a completion event.

The engine now processes events in time order. Most are ordinary completions, each freeing a core, starting the next queued task, and possibly stealing work from a busier machine. Interspersed are interruption events drawn from the scenario. Each one suspends a machine, stops its billing, checkpoints all its tasks, and pushes each task through the five step cascade. When the surviving machines cannot absorb the displaced work in the time remaining, the deadline aware rule buys exactly enough new spot capacity to close the gap, and those machines become billable the moment they come online.

The run ends when the event queue empties. The makespan is the last completion time, the cost is the sum over machines of price multiplied by billed seconds, and the deadline misses are counted against each task's own deadline.

12. Parameter reference

Parameter	Value	What it controls
alpha	0.5	Balance between cost and time in the fitness function
max iterations	200	Length of the outer search
max attempts	50	Tries per local search pass
swap rate	0.10	Fraction of tasks moved per attempt
max failed	20	Stalled iterations before the bound is relaxed
relaxed rate	0.25	Size of each relaxation, capped at the deadline
burst rate	0.2	Burstable machines as a fraction of those selected
allocation cycle	900 s	Idle time before a machine is terminated
startup latency	45 s	Time before a new machine is usable
checkpoint overhead	0.10	Cost of periodically saving progress
risk threshold	0.05	Minimum risk to consider a preemptive replacement
max per event	10	Cap on machines bought in one reaction
deadline factor	1.0	Multiplier on the derived deadline
kappa	3	Deadline as a multiple of the perfect packing bound
seeds	30	Repetitions per configuration